

Borrowed Badge — Tech Stack & Workflow

Project: borrowed-badge — a pre-payment verifier for Indian retail investors **Analysed:** 2026-10-04 **Scope:** verified against the actual source in the repo, not just the README

A badge can be real without the person wearing it being its owner. The tool answers whether a channel (website, UPI/bank, phone) is actually attached to the entity that holds a displayed SEBI registration — not merely whether the registration number looks valid.

1. Tech Stack

Layer	Technology
Backend	Python 3.12 + FastAPI (main.py), served by Uvicorn
Data models	Pydantic v2 (schemas.py) for request/response; dataclasses for internal types
Fuzzy matching	stdlib difflib with an optional upgrade to rapidfuzz (detected at import, falls back cleanly) — textmatch.py
External data	RDAP (rdap.org/domain/) for live domain registration facts, via stdlib urllib — no third-party HTTP client
Frontend	Single-page vanilla HTML/JS (frontend/index.html), no build step, no framework — plain fetch() against the API
Testing	pytest + httpx (for fastapi.testclient)
Config	Env-var-overridable dataclass settings in config.py (BB_* vars)

The dependency set is deliberately small — runtime deps are just FastAPI, Uvicorn and Pydantic (plus optional rapidfuzz). **No database, no ORM, no ML libraries.**

2. Workflow / Architecture

A **stateless, six-module pipeline** that answers one question: *is this channel actually attached to the entity that holds the displayed SEBI registration?*

Request flow (POST /verify → verdict_engine)

- Anchor** — entity_reference.py (Module A) Builds an Entity from details the user **read off SEBI's own listing** (name, status, official website / UPI / phone). The tool holds **no SEBI data**. An Entity.richness map records which channel types the anchor actually pins down.
- Channels** — run only if supplied: - domain_analyzer.py (B) — reverse-link check (fetch the listed site, look for a link back) + live RDAP facts + local lookalike / homoglyph / TLD reasoning -

`payment_analyzer.py` (C) — UPI handle, bank / IFSC, payee-name correspondence -
`phone_analyzer.py` (D) — presence in the anchor + number "shape" (1600-series vs plain mobile, as a risk flag only)

3. **Behaviour flags** — `behaviour_flags.py` (E) 12 transparent regex rules over pasted text (guaranteed returns, VIP groups, personal-account payment, etc.). Reported **alongside** the verdict, never merged into it.
4. **Verdict engine** — `verdict_engine.py` (F) Deterministic, no model. Two modes: - **Anchored** (SEBI details supplied): worst-per-channel verdict, with entity-level overrides (`ENTITY_NOT_ACTIVE` and name-contradiction → `MISMATCH`). - **Unanchored**: reports live facts plus a risk level, and states plainly that only SEBI can confirm the registration.

Verdicts (`types.py`)

Severity order, worst wins:

```
CONNECTED < CANNOT_VERIFY < NOT_CONNECTED < MISMATCH
```

The central design rule: `NOT_CONNECTED` is only claimed when the anchor is rich enough that a real link *would* have shown up — otherwise `CANNOT_VERIFY`, so honest small firms are not falsely accused.

Endpoints in `backend/main.py`

Method	Path	Purpose
GET	<code>/health</code>	Liveness + dataset / source metadata
POST	<code>/verify</code>	Run a check, return a full explainable verdict
GET	<code>/</code>	Serve the single-page frontend

3. Important Note: README vs. Code Drift

The **README describes an older version (v1 "seeded mock")**; the code is **v2.0.0 ("live, stateless")**.

Worth knowing before trusting either document:

- The README documents a `data/` folder (`mock_registry.json`, `domain_intel.json`), `scripts/generate_*.py`, `backend/registry_anchor.py`, `backend/demos.py`, a `BB_ENABLE_LIVE_LOOKUPS` flag, and a 94-case eval suite. **None of those exist on disk** — only their stale `.pyc` files remain in `__pycache__`. The real code uses `entity_reference.py`, live RDAP, and no dataset at all.
- **Tests are mostly gone as source.** `tests/` contains only `test_behaviour_flags.py`; the API / domain / payment / verdict / regression test files survive only as compiled `.pyc`.
- **The frontend calls endpoints that don't exist.** `index.html` fetches `/search?q=` and `/demo-scenarios`, but `main.py` only defines `/health`, `/verify` and `/`. Those UI paths will return 404.

Generated from a direct read of the repository source.